

GRA4158 Marketing - Analysis of Experiments, Portfolio 3

Vilijam Cekov, Luiz Felipe De Souza Simao

Table of contents

1	Import Data	2
2	Estimate the treatment effect using simple linear Regression Adjustment	3
3	Propensity Score Estimation and Overlap Diagnostic	4
4	Matching comparisons	6
4.1	1:1 Nearest Neighbor Matching WITHOUT Replacement	6
4.2	1:1 Nearest Neighbor Matching WITH Replacement	9
4.3	Euclidean Distance Matching	13
4.3.1	Comparing matching quality across methods	15
5	Inverse Probability Weighting (IPW)	16
5.1	IPW Critical Discussion Points	18
6	Augmented Inverse Probability Weighting (AIPW) - Manual Estimation	18
6.1	AIPW Critical Discussion Points	20
7	Double Machine Learning (DML): PLR with Cross-Fitting	21
7.1	DML Discussion Points	25
8	Comparison of Treatment Effects Across Methods	26
9	Discussion	30
9.1	1. Comparison of ATE vs. ATT Estimates	30
9.2	2. Methodological Divergence and Instability	30
9.3	3. The Role of Double Robustness and Machine Learning	31
9.4	4. Conclusion: Which Estimate to Trust?	31
9.4.1	Suggestions for the Marketing Manager	31

1 Import Data

```
casedata_task3 <- read.csv("CaseData_Task3.csv", stringsAsFactors = FALSE)

# Display the structure of the data to verify column names
str(casedata_task3)
```

```
'data.frame':  5000 obs. of  22 variables:
 $ X1 : num  23.8 21 22.8 25.8 22.9 24.9 24.1 21.4 24.8 23.8 ...
 $ X2 : num  13.7 14.4 11.5 13.5 13.6 12.9 11.8 12.4 14.2 15.9 ...
 $ X3 : num  22.5 21.5 20.6 22.7 21.6 19.4 22.9 19.5 21.6 23.4 ...
 $ X4 : num  16.2 14.5 14.7 15.8 17.5 13.8 17 13.8 15.5 16.8 ...
 $ X5 : int   0 0 0 1 1 0 1 0 0 1 ...
 $ X6 : num  22.9 23.4 20.2 22.5 23.5 21.1 24.6 20 23.9 20 ...
 $ X7 : num  11.6 11.2 9.6 11.5 12.7 9.6 12.9 9.5 11.4 9.2 ...
 $ X8 : num  10.1 12.8 9.7 10.3 12.3 6.3 9.6 7.4 10.6 11.9 ...
 $ X9 : num  18.9 20.2 19 21.9 19.7 15 20.2 17.1 23.8 20.2 ...
 $ X10: num  14.6 13 15.1 18.1 13.7 11.4 15.2 10.3 19.7 14.2 ...
 $ X11: num  13.1 13.7 14.6 14.2 14.5 12 14.1 11.4 16.4 13.6 ...
 $ X12: num   6.2 5.5 6.6 5.3 6.7 3.8 5.9 5 8.8 6.5 ...
 $ X13: num  14 13.5 16.6 13.3 14.8 11.8 13.8 12.9 15.2 15.3 ...
 $ X14: num   5.4 3.7 7.3 4.1 4.2 4.4 4.5 3.7 5.1 5.4 ...
 $ X15: num  11.6 9 13.4 10.2 10.5 10.6 11.8 9.6 11.8 11.7 ...
 $ X16: num  10.9 9.9 12.4 10.6 10.2 9.2 11.2 8.9 11 11.7 ...
 $ X17: num  12 12.4 13.3 11.1 11.8 10.7 12.6 12 11.7 12.9 ...
 $ X18: num   8.6 7.4 8.1 5.9 8.5 6.8 7.5 7.1 6.6 8.1 ...
 $ X19: num  12.3 12.1 11.8 10.3 12.8 10.2 11.1 10.8 11 12 ...
 $ X20: int   0 0 1 1 0 1 0 0 1 0 ...
 $ y  : int  1463 2410 659 2202 2196 2329 1993 2467 1644 1499 ...
 $ d  : int   0 1 1 1 0 1 1 1 0 1 ...
```

```
# Check the first few rows
head(casedata_task3)
```

	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	X12	X13	X14	X15	X16
1	23.8	13.7	22.5	16.2	0	22.9	11.6	10.1	18.9	14.6	13.1	6.2	14.0	5.4	11.6	10.9
2	21.0	14.4	21.5	14.5	0	23.4	11.2	12.8	20.2	13.0	13.7	5.5	13.5	3.7	9.0	9.9

```

3 22.8 11.5 20.6 14.7 0 20.2 9.6 9.7 19.0 15.1 14.6 6.6 16.6 7.3 13.4 12.4
4 25.8 13.5 22.7 15.8 1 22.5 11.5 10.3 21.9 18.1 14.2 5.3 13.3 4.1 10.2 10.6
5 22.9 13.6 21.6 17.5 1 23.5 12.7 12.3 19.7 13.7 14.5 6.7 14.8 4.2 10.5 10.2
6 24.9 12.9 19.4 13.8 0 21.1 9.6 6.3 15.0 11.4 12.0 3.8 11.8 4.4 10.6 9.2
      X17 X18 X19 X20      y d
1 12.0 8.6 12.3 0 1463 0
2 12.4 7.4 12.1 0 2410 1
3 13.3 8.1 11.8 1 659 1
4 11.1 5.9 10.3 1 2202 1
5 11.8 8.5 12.8 0 2196 0
6 10.7 6.8 10.2 1 2329 1

```

2 Estimate the treatment effect using simple linear Regression Adjustment

The estimate for the discount coefficient is the Regression Adjustment ATE

```

model_adj <- lm(y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 +
               X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20,
               data = casedata_task3)

coeftest(model_adj)["d", , drop = FALSE]

```

```

      Estimate Std. Error t value Pr(>|t|)
d 15.15365     5.704223 2.656566 0.007919282

```

Extract the ATE value from the regression and calculate robust standard errors using the HC3 estimator, which is more reliable in small samples and with heteroskedasticity.

```
coeftest(model_adj, vcov = vcovHC(model_adj, type = "HC3"))
```

t test of coefficients:

```

              Estimate Std. Error t value Pr(>|t|)
(Intercept) 4697.04561    76.15859  61.6745 < 2.2e-16 ***
d             15.15365     5.80286   2.6114  0.009044 **
X1              3.06184     1.51318   2.0235  0.043080 *
X2             -1.56049     1.81397  -0.8603  0.389687

```

X3	1.36316	1.83034	0.7448	0.456453
X4	-2.84763	1.74765	-1.6294	0.103291
X5	36.24104	5.67533	6.3857	1.861e-10 ***
X6	-1.72139	1.71427	-1.0042	0.315354
X7	21.87065	1.79930	12.1551	< 2.2e-16 ***
X8	-1.31713	1.85035	-0.7118	0.476607
X9	0.87561	1.80426	0.4853	0.627484
X10	-1.33908	1.86892	-0.7165	0.473716
X11	-0.10716	3.77342	-0.0284	0.977345
X12	1.27229	4.16008	0.3058	0.759745
X13	-7.17543	3.72007	-1.9288	0.053807 .
X14	-304.00261	3.64002	-83.5167	< 2.2e-16 ***
X15	-133.66096	3.78412	-35.3216	< 2.2e-16 ***
X16	0.51387	3.54850	0.1448	0.884863
X17	1.52685	3.65959	0.4172	0.676537
X18	-1.88837	3.63423	-0.5196	0.603362
X19	0.87427	3.36898	0.2595	0.795255
X20	-26.96513	5.41454	-4.9801	6.569e-07 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

The estimated Average Treatment Effect (ATE) for the treatment is: \$15.15

3 Propensity Score Estimation and Overlap Diagnostic

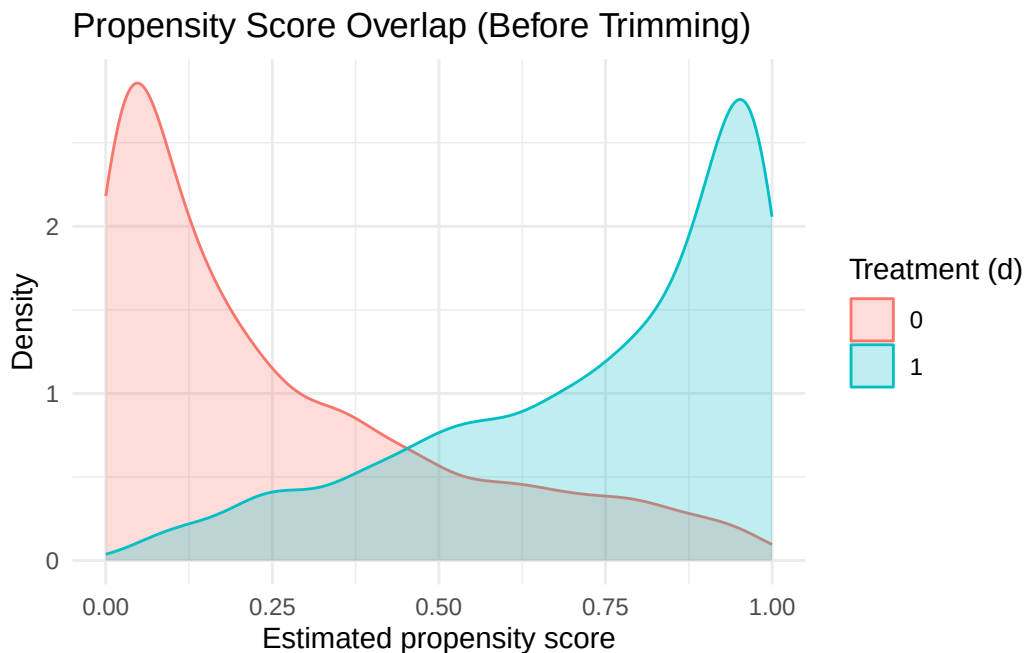
Before implementing matching or weighting, I first estimate the propensity scores and evaluate the common support assumption. This step is a necessary prerequisite to ensure that there is sufficient overlap in the covariate distributions between the treated and control groups

```
# Complete-case sample for IPW variables
ipw_vars <- c("y", "d", paste0("X", 1:20))
df_ipw <- casedata_task3[complete.cases(casedata_task3[, ipw_vars]), ]

# 1) Estimate propensity scores with logistic regression
ps_model <- glm(
  d ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 +
    X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20,
  data = df_ipw,
  family = binomial(link = "logit")
)
```

```
df_ipw$ps <- predict(ps_model, type = "response")

# Overlap/common support diagnostic BEFORE trimming
ggplot(df_ipw, aes(x = ps, color = factor(d), fill = factor(d))) +
  geom_density(alpha = 0.25) +
  labs(
    title = "Propensity Score Overlap (Before Trimming)",
    x = "Estimated propensity score",
    y = "Density",
    color = "Treatment (d)",
    fill = "Treatment (d)"
  ) +
  theme_minimal()
```



Discussion of Overlap Diagnostic: The density plot of the estimated propensity scores shows that the treated group ($d=1$) has a distribution of scores that is quite different from the control group ($d=0$). The treated units tend to have higher propensity scores, while many control units have scores close to zero. This indicates poor overlap between the groups, which can lead to issues with both matching and weighting methods. In particular, IPW may produce extreme weights for control units with very low propensity scores, leading to unstable estimates. Matching may also struggle to find good matches for treated units if the control group is not well represented across the range of propensity scores. This is a red flag that we may need to consider trimming the sample or using more robust methods that are less

sensitive to poor overlap, such as AIPW or DML. As such, it is important to consider that the limited overlap is why both matching and IPW weights may struggle with this dataset, and why we should be cautious in interpreting results from those methods.

4 Matching comparisons

Preparing the data for matching exercises

```
ps_formula <- d ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 + X11 +  
X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20
```

4.1 1:1 Nearest Neighbor Matching WITHOUT Replacement

```
match_no_replace <- matchit(ps_formula,  
                             data = casedata_task3,  
                             method = "nearest",  
                             distance = "glm",    # Use logistic reg for scores  
                             replace = FALSE,     # Without replacement  
                             ratio = 1)          # 1:1 matching  
  
# View a summary to check covariate balance (ideally Std. Mean Diff < 0.1)  
print(summary(match_no_replace, un = FALSE))
```

Call:

```
matchit(formula = ps_formula, data = casedata_task3, method = "nearest",  
        distance = "glm", replace = FALSE, ratio = 1)
```

Summary of Balance for Matched Data:

	Means Treated	Means Control	Std. Mean Diff.	Var. Ratio	eCDF Mean
distance	0.7342	0.2657	1.9201	0.9190	0.3878
X1	24.0591	24.0637	-0.0023	0.9305	0.0067
X2	14.1096	14.0910	0.0093	0.9578	0.0048
X3	22.0102	22.0009	0.0047	0.9816	0.0027
X4	15.7842	15.7595	0.0125	0.9536	0.0041
X5	0.5417	0.5324	0.0187	.	0.0093
X6	23.0879	23.0536	0.0173	0.9687	0.0039
X7	11.4389	11.3466	0.0462	1.0201	0.0073
X8	10.4588	10.2308	0.1115	1.0419	0.0180

X9	20.1062	20.4062	-0.1476	0.9862	0.0230
X10	14.7843	15.6086	-0.4177	1.0232	0.0640
X11	13.7279	14.4796	-0.8253	0.9879	0.1073
X12	5.7278	6.8929	-1.5370	1.0032	0.1738
X13	13.9400	14.7616	-0.9285	0.9906	0.1173
X14	4.9429	5.5103	-0.5895	1.0138	0.0846
X15	11.4658	11.8530	-0.3924	1.0262	0.0570
X16	11.3215	11.5974	-0.2730	1.0655	0.0386
X17	12.3863	12.5703	-0.1844	1.0055	0.0270
X18	7.8680	8.0111	-0.1386	1.0906	0.0202
X19	11.5044	11.5751	-0.0687	1.0678	0.0108
X20	0.3464	0.3493	-0.0060	.	0.0028

eCDF Max Std. Pair Dist.

distance	0.6293	1.9201
X1	0.0259	1.1445
X2	0.0158	1.1531
X3	0.0105	1.1438
X4	0.0138	1.1503
X5	0.0093	0.9881
X6	0.0170	1.1274
X7	0.0251	1.1152
X8	0.0559	1.1007
X9	0.0713	1.1326
X10	0.1848	1.1405
X11	0.3205	1.2056
X12	0.5543	1.5408
X13	0.3553	1.2431
X14	0.2459	1.1738
X15	0.1572	1.1387
X16	0.1155	1.1115
X17	0.0729	1.1178
X18	0.0583	1.0849
X19	0.0344	1.0910
X20	0.0028	0.9580

Sample Sizes:

	Control	Treated
All	2532	2468
Matched	2468	2468
Unmatched	64	0
Discarded	0	0

```
# Extract the matched dataset
data_matched_no_replace <- match.data(match_no_replace)

# Estimate the Treatment Effect
# We include the covariates in the final model to make it "doubly robust"
model_no_replace <- lm(y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 +
                      X10 + X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 +
                      X19 + X20,
                      data = data_matched_no_replace,
                      weights = weights)
```

Looking at the balance table, the results are not great. Many covariates still have SMDs well above 0.1 – X12 is at -1.54 , X11 at -0.83 , X13 at -0.93 . The propensity score distance SMD is 1.92, which is quite high. So even after matching, the treated and control groups still look pretty different on these variables. We think this happens because without replacement, once a good control unit gets matched it's gone, and later treated units get stuck with worse matches. This makes us less confident in the ATT from this specification.

Extract the ATT value and calculate robust standard errors using Variance/Covariance Heteroscedasticity-Consistent matrix estimation, which is used to adjust the standard errors of the estimated treatment effects to account for the matched data structure and potential heteroscedasticity.

```
# Calculate robust standard errors
print(coeftest(model_no_replace, vcov. = vcovHC))
```

t test of coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	4662.91246	76.89579	60.6394	< 2.2e-16	***
d	17.15026	5.83088	2.9413	0.003284	**
X1	3.06093	1.51975	2.0141	0.044053	*
X2	-1.43813	1.82390	-0.7885	0.430447	
X3	1.29676	1.84075	0.7045	0.481171	
X4	-2.86753	1.75898	-1.6302	0.103118	
X5	36.29551	5.71131	6.3550	2.272e-10	***
X6	-1.51569	1.71731	-0.8826	0.377500	
X7	21.90153	1.80833	12.1115	< 2.2e-16	***
X8	-1.77083	1.86226	-0.9509	0.341699	
X9	0.87361	1.80706	0.4834	0.628801	
X10	-1.51174	1.88125	-0.8036	0.421678	

X11	0.87193	3.79489	0.2298	0.818284
X12	2.95103	4.24587	0.6950	0.487066
X13	-6.47073	3.73861	-1.7308	0.083553 .
X14	-304.35470	3.65250	-83.3277	< 2.2e-16 ***
X15	-133.08910	3.80349	-34.9913	< 2.2e-16 ***
X16	0.77594	3.58534	0.2164	0.828669
X17	1.03521	3.69715	0.2800	0.779487
X18	-1.56797	3.65007	-0.4296	0.667525
X19	0.66176	3.39599	0.1949	0.845507
X20	-27.64769	5.43725	-5.0849	3.816e-07 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

The estimated Average Treatment Effect on the Treated for the treatment is (ATT): \$17.15

4.2 1:1 Nearest Neighbor Matching WITH Replacement

```
match_with_replace <- matchit(ps_formula,
                              data = casedata_task3,
                              method = "nearest",
                              distance = "glm",
                              replace = TRUE,      # With replacement
                              caliper = 0.2,      # Cap match distance at 0.2 SDs of PS
                              std.caliper = TRUE,
                              ratio = 1)

# Balance Summary (With Replacement)
print(summary(match_with_replace, un = FALSE))
```

Call:

```
matchit(formula = ps_formula, data = casedata_task3, method = "nearest",
        distance = "glm", replace = TRUE, caliper = 0.2, std.caliper = TRUE,
        ratio = 1)
```

Summary of Balance for Matched Data:

	Means Treated	Means Control	Std. Mean Diff.	Var. Ratio	eCDF Mean
distance	0.7342	0.7341	0.0005	0.9942	0.0012
X1	24.0591	24.0335	0.0129	0.8633	0.0137
X2	14.1096	14.0685	0.0206	0.8747	0.0153

X3	22.0102	22.0362	-0.0130	0.8053	0.0165
X4	15.7842	15.7235	0.0308	0.9084	0.0116
X5	0.5417	0.5211	0.0415	.	0.0207
X6	23.0879	22.8316	0.1293	0.9564	0.0208
X7	11.4389	10.9902	0.2247	1.1376	0.0358
X8	10.4588	9.9677	0.2401	1.1778	0.0396
X9	20.1062	19.7199	0.1901	0.9503	0.0292
X10	14.7843	14.5226	0.1326	1.0423	0.0210
X11	13.7279	13.7266	0.0014	1.1714	0.0111
X12	5.7278	5.6479	0.1055	0.8326	0.0147
X13	13.9400	13.8658	0.0838	1.0795	0.0146
X14	4.9429	4.8341	0.1131	0.9741	0.0167
X15	11.4658	11.3919	0.0749	1.0047	0.0125
X16	11.3215	11.3739	-0.0518	0.9714	0.0117
X17	12.3863	12.3467	0.0398	0.8209	0.0143
X18	7.8680	7.7625	0.1022	0.8899	0.0170
X19	11.5044	11.5057	-0.0013	1.0171	0.0116
X20	0.3464	0.3343	0.0255	.	0.0122

eCDF Max Std. Pair Dist.

distance	0.0425	0.0027
X1	0.0506	1.1746
X2	0.0523	1.1719
X3	0.0434	1.2037
X4	0.0442	1.1883
X5	0.0207	0.9946
X6	0.0754	1.1502
X7	0.1333	1.1094
X8	0.1264	1.1154
X9	0.1017	1.1269
X10	0.0940	1.0668
X11	0.0425	0.9713
X12	0.0604	0.5051
X13	0.0600	0.9414
X14	0.0802	1.0369
X15	0.0758	1.0793
X16	0.0506	1.1028
X17	0.0409	1.1375
X18	0.0681	1.1460
X19	0.0470	1.1050
X20	0.0122	0.9179

Sample Sizes:

Control Treated

All	2532.	2468
Matched (ESS)	139.53	2468
Matched	707.	2468
Unmatched	1825.	0
Discarded	0.	0

Balance is much better here. Most SMDs drop below 0.13, and the propensity score distance SMD is basically 0 (0.0005). X7 and X8 are still a bit above 0.1 (~0.22–0.24), but overall this is a big improvement over the without-replacement version. This makes sense – allowing replacement means each treated unit can grab the best available control, even if that control has already been used. The downside is that some controls get reused a lot, which is why we need to cluster the standard errors by subclass below.

Extract the ATT value and calculate robust standard errors clustered by the matched pair ID (`subclass`), which accounts for the fact that some control units may be used multiple times as matches, leading to correlated errors within matched pairs.

```
# Extract the matched dataset
data_matched_with_replace <- get_matches(match_with_replace)

# Estimate the Treatment Effect
model_with_replace <- lm(y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 +
                        X10 + X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 +
                        X19 + X20,
                        data = data_matched_with_replace,
                        weights = weights)
print(coefest(model_with_replace, vcov. = vcovCL, cluster = ~subclass))
```

t test of coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	4397.39206	75.72990	58.0668	< 2.2e-16	***
d	23.25309	4.38925	5.2977	1.224e-07	***
X1	3.03662	1.44236	2.1053	0.0353144	*
X2	-2.50465	1.83192	-1.3672	0.1716165	
X3	3.88159	1.83070	2.1203	0.0340329	*
X4	-1.66259	1.80731	-0.9199	0.3576577	
X5	59.55033	5.81260	10.2450	< 2.2e-16	***
X6	-5.88580	1.74421	-3.3745	0.0007453	***
X7	18.50881	1.87071	9.8940	< 2.2e-16	***
X8	3.07111	1.91503	1.6037	0.1088472	

X9	0.44156	1.73320	0.2548	0.7989141	
X10	-3.07928	1.86650	-1.6498	0.0990561	.
X11	11.14108	3.66045	3.0436	0.0023497	**
X12	-16.28603	3.85220	-4.2277	2.404e-05	***
X13	4.06409	3.65274	1.1126	0.2659291	
X14	-303.81083	3.75733	-80.8582	< 2.2e-16	***
X15	-116.73137	4.03738	-28.9126	< 2.2e-16	***
X16	-5.59644	3.63604	-1.5392	0.1238302	
X17	0.87081	3.56892	0.2440	0.8072427	
X18	4.16273	3.34672	1.2438	0.2136242	
X19	-2.06649	3.37233	-0.6128	0.5400503	
X20	-27.46830	5.55989	-4.9404	8.053e-07	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

The estimated Average Treatment Effect on the Treated for the treatment is (ATT): \$23.25

Assessing the Need for Regression Adjustment To evaluate the redundancy of including covariates after matching, I compared the “doubly robust” model (which includes all 20 covariates in the final regression) against a “simple matched model” that uses only the treatment indicator on the matched sample. Discussion of Results: If the matching has achieved high balance (as indicated by my SMD diagnostics showing most variables under 0.13), the estimates from these two models should be nearly identical. A significant difference would suggest that matching was imperfect and the regression adjustment is still working to “clean up” remaining imbalance in the covariates

```
# Simple matched model without covariates
model_simple_matched <- lm(y ~ d,
                           data = data_matched_with_replace,
                           weights = weights)

# Summary of the simple model
coeftest(model_simple_matched)["d", , drop = FALSE]
```

	Estimate	Std. Error	t value	Pr(> t)
d	-10.32942	11.72173	-0.8812191	0.3782422

Simple Matched ATT: \$-10.33

Doubly Robust ATT: \$23.25

As made evident by the highly inaccurate ATT measure of the simple matched model, the simple matched model's ATT is quite different from the doubly robust version (-\$10.30 vs. \$23.30), which suggests that the matching alone didn't achieve good balance and the regression adjustment is still doing important work to control for confounding.

4.3 Euclidean Distance Matching

```
euc_out <- matchit(
  d ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 +
      X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20,
  data = casedata_task3,
  method = "nearest",
  distance = "euclidean",
  normalize = TRUE
)

summary(euc_out)
```

Call:

```
matchit(formula = d ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 +
  X9 + X10 + X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 +
  X19 + X20, data = casedata_task3, method = "nearest", distance = "euclidean",
  normalize = TRUE)
```

Summary of Balance for All Data:

	Means Treated	Means Control	Std. Mean Diff.	Var. Ratio	eCDF Mean	eCDF Max
X1	24.0591	24.0646	-0.0028	0.9359	0.0063	0.0248
X2	14.1096	14.0950	0.0073	0.9616	0.0049	0.0155
X3	22.0102	22.0011	0.0046	0.9769	0.0029	0.0106
X4	15.7842	15.7626	0.0109	0.9435	0.0044	0.0134
X5	0.5417	0.5336	0.0164	.	0.0082	0.0082
X6	23.0879	23.0587	0.0147	0.9675	0.0036	0.0147
X7	11.4389	11.3458	0.0467	1.0213	0.0073	0.0240
X8	10.4588	10.2257	0.1140	1.0445	0.0184	0.0577
X9	20.1062	20.4252	-0.1570	0.9849	0.0245	0.0728
X10	14.7843	15.6430	-0.4351	1.0106	0.0667	0.1897
X11	13.7279	14.5120	-0.8609	0.9437	0.1119	0.3290
X12	5.7278	6.9410	-1.6004	0.8800	0.1809	0.5601
X13	13.9400	14.7973	-0.9687	0.9357	0.1224	0.3611

X14	4.9429	5.5363	-0.6165	0.9893	0.0884	0.2546
X15	11.4658	11.8701	-0.4097	1.0171	0.0595	0.1626
X16	11.3215	11.6137	-0.2891	1.0480	0.0408	0.1214
X17	12.3863	12.5792	-0.1933	0.9952	0.0283	0.0740
X18	7.8680	8.0182	-0.1455	1.0856	0.0212	0.0596
X19	11.5044	11.5839	-0.0772	1.0663	0.0121	0.0381
X20	0.3464	0.3468	-0.0007	.	0.0003	0.0003

Summary of Balance for Matched Data:

	Means Treated	Means Control	Std. Mean Diff.	Var. Ratio	eCDF Mean	eCDF Max
X1	24.0591	24.0653	-0.0032	0.9349	0.0064	0.0243
X2	14.1096	14.0893	0.0102	0.9621	0.0045	0.0158
X3	22.0102	21.9816	0.0144	0.9787	0.0034	0.0122
X4	15.7842	15.7383	0.0232	0.9592	0.0046	0.0162
X5	0.5417	0.5304	0.0228	.	0.0113	0.0113
X6	23.0879	23.0280	0.0302	0.9705	0.0052	0.0211
X7	11.4389	11.3074	0.0659	1.0282	0.0103	0.0320
X8	10.4588	10.1615	0.1453	1.0810	0.0232	0.0681
X9	20.1062	20.3464	-0.1182	1.0326	0.0185	0.0608
X10	14.7843	15.5527	-0.3893	1.0847	0.0597	0.1771
X11	13.7279	14.4667	-0.8111	1.0295	0.1055	0.3201
X12	5.7278	6.9021	-1.5492	0.9611	0.1752	0.5543
X13	13.9400	14.7636	-0.9307	0.9911	0.1176	0.3553
X14	4.9429	5.5066	-0.5856	1.0280	0.0841	0.2464
X15	11.4658	11.8451	-0.3844	1.0409	0.0558	0.1540
X16	11.3215	11.5895	-0.2652	1.0770	0.0375	0.1126
X17	12.3863	12.5609	-0.1750	1.0136	0.0256	0.0701
X18	7.8680	8.0048	-0.1326	1.0949	0.0193	0.0555
X19	11.5044	11.5732	-0.0668	1.0679	0.0105	0.0340
X20	0.3464	0.3509	-0.0094	.	0.0045	0.0045

Std. Pair Dist.

X1	0.4711
X2	0.4388
X3	0.4435
X4	0.4554
X5	0.5156
X6	0.4568
X7	0.4445
X8	0.4594
X9	0.4739
X10	0.6054
X11	1.0269
X12	1.6159

X13	1.1450
X14	0.8978
X15	0.7994
X16	0.7349
X17	0.7004
X18	0.6556
X19	0.7062
X20	0.6667

Sample Sizes:

	Control	Treated
All	2532	2468
Matched	2468	2468
Unmatched	64	0
Discarded	0	0

```
# bal.tab(euc_out) - balance info already shown in summary above

m_data_euc <- match.data(euc_out)
lm_euc <- lm(y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 + X11 +
            X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20,
            data = m_data_euc, weights = weights)
```

Extract the ATT value and calculate robust standard errors using the HC3 estimator

```
coeftest(lm_euc)["d", , drop = FALSE]
```

	Estimate	Std. Error	t value	Pr(> t)
d	16.15806	5.71602	2.826802	0.004720542

```
# Robust Standard Errors (Euclidean Matching) - treatment row only
print(coeftest(lm_euc, vcov. = vcovHC(lm_euc, type = "HC3"))["d", , drop = FALSE])
```

	Estimate	Std. Error	t value	Pr(> t)
d	16.15806	5.802823	2.784516	0.00538128

4.3.1 Comparing matching quality across methods

Looking at the balance tables side by side, Euclidean distance matching and PSM without replacement perform similarly – and both are pretty bad. The Euclidean SMDs are actually

slightly worse on the key problem covariates ($X_{12} = -1.60$ vs. -1.54 for PSM, $X_{11} = -0.86$ vs. -0.83). Neither method manages to bring X_{10} – X_{15} anywhere close to the 0.1 threshold. PSM with replacement is clearly the best of the three – it gets most covariates under 0.13 and the propensity score distance is essentially perfectly balanced. So in terms of matching quality, with-replacement PSM wins by a wide margin here, though it comes at the cost of reusing control units and needing clustered standard errors. The poor balance in the other two methods means their ATT estimates should probably be taken with a grain of salt.

On common support: the sample sizes tables show that 64 control units (out of 2532) went unmatched, meaning they had no reasonably close treated counterpart. No treated units were discarded. So we're not losing a huge chunk of the data, which is reassuring – but the fact that those 64 controls couldn't find a match does hint at the limited overlap between the groups, which is also visible in the propensity score density plot in the IPW section. The separation in that plot explains why both the matching balance and the IPW weights struggle here.

5 Inverse Probability Weighting (IPW)

```
# 2) Trim propensity scores to reduce extreme weights under weak overlap
df_ipw$ps_trim <- pmin(pmax(df_ipw$ps, 0.01), 0.99)

# Function requested in prompt: two-step IPW ATE with logistic PS + trimming
estimate_ipw_ate <- function(data, trim = 0.01) {
  ps_fit <- glm(
    d ~ X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 +
      X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20,
    data = data,
    family = binomial(link = "logit")
  )

  ps_hat <- predict(ps_fit, newdata = data, type = "response")
  ps_hat <- pmin(pmax(ps_hat, trim), 1 - trim)

  mean(data$y * (data$d / ps_hat - (1 - data$d) / (1 - ps_hat)))
}

# 3) Manual IPW ATE using slide formula:
# ATE = mean( y * ( d/ps - (1-d)/(1-ps) ) )
ate_ipw_manual <- estimate_ipw_ate(df_ipw, trim = 0.01)

cat(sprintf("\nManual IPW ATE (primary estimate): %.2f\n", ate_ipw_manual))
```


Manual IPW ATE (primary estimate): \$-318.12

```
# 4) Bootstrap inference for the full two-step IPW estimator:
#   re-sample rows, re-estimate propensity model, re-calculate ATE each draw.
set.seed(123)
B_ipw <- 1000
n_ipw <- nrow(df_ipw)
boot_ipw_ate <- numeric(B_ipw)

for (b in seq_len(B_ipw)) {
  idx <- sample.int(n_ipw, size = n_ipw, replace = TRUE)
  boot_data <- df_ipw[idx, ]
  boot_ipw_ate[b] <- estimate_ipw_ate(boot_data, trim = 0.01)
}

ipw_boot_mean <- mean(boot_ipw_ate)
ipw_boot_se <- sd(boot_ipw_ate)
ipw_boot_ci <- quantile(boot_ipw_ate, probs = c(0.025, 0.975))
```

Table 1: IPW Bootstrap Results

Statistic	Value
Bootstrap Mean (IPW)	-315.5704
Bootstrap SE (IPW)	81.2465
95% Percentile CI (IPW)	[-487.5767, -173.1673]

```
# 5) Secondary comparison: IPW weighted regression-adjustment model
df_ipw$ipw_weight <- ifelse(df_ipw$d == 1,
  1 / df_ipw$ps_trim,
  1 / (1 - df_ipw$ps_trim))

# Inverse Probability Weights Summary:
summary(df_ipw$ipw_weight)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
1.010	1.059	1.216	1.972	1.708	100.000

```
#Secondary comparison: weighted lm IPW-RA model
```

```
lm_ipw <- lm(y ~ d + X1 + X2 + X3 + X4 + X5 + X6 + X7 + X8 + X9 + X10 + X11 +
            X12 + X13 + X14 + X15 + X16 + X17 + X18 + X19 + X20,
            data = df_ipw, weights = ipw_weight)
```

```
print(coeftest(lm_ipw)["d", , drop = FALSE])
```

```
      Estimate Std. Error  t value    Pr(>|t|)
d 15.80639     4.310274  3.667142 0.000247837
```

```
print(coeftest(lm_ipw, vcov. = vcovHC(lm_ipw, type = "HC3"))["d", , drop = FALSE])
```

```
      Estimate Std. Error t value    Pr(>|t|)
d 15.80639     8.304334  1.90339 0.05704728
```

5.1 IPW Critical Discussion Points

The manual IPW estimate came out at around $-\$318$, which obviously doesn't make sense for a sales effect and is likely due to its sensitivity to poor overlap. The weighted regression version ($\sim \$15.81$) does much better because the regression component doesn't rely purely on the weights. It also shows why AIPW and DML are generally more appealing: they don't put all their eggs in the weighting basket.

6 Augmented Inverse Probability Weighting (AIPW) - Manual Estimation

This section implements AIPW manually using the doubly robust score:

$$ATE = \frac{1}{N} \sum_{i=1}^N \left(\frac{T_i}{\hat{p}(X_i)} (Y_i - \hat{\mu}_{T=1}(X_i)) + \hat{\mu}_{T=1}(X_i) \right) - \frac{1}{N} \sum_{i=1}^N \left(\frac{(1-T_i)}{1-\hat{p}(X_i)} (Y_i - \hat{\mu}_{T=0}(X_i)) + \hat{\mu}_{T=0}(X_i) \right)$$

```
df <- casedata_task3
df$treatment <- df$d
df$sales <- df$y
covariates <- paste0("X", 1:20)

# Basic checks
```

```

stopifnot(all(c("treatment", "sales") %in% names(df)))
stopifnot(all(covariates %in% names(df)))
stopifnot(all(na.omit(unique(df$treatment)) %in% c(0, 1)))

# Keep complete cases for variables used in AIPW
aipw_vars <- c("treatment", "sales", covariates)
df_aipw <- df[complete.cases(df[, aipw_vars]), aipw_vars]

# AIPW function returning the sample ATE using the manual formula
estimate_aipw_ate <- function(data, covariate_names, trim = 0.01) {
  # 1) Selection model (propensity score): P(T=1|X)
  ps_formula <- as.formula(
    paste("treatment ~", paste(covariate_names, collapse = " + "))
  )
  ps_model <- glm(ps_formula, data = data, family = binomial(link = "logit"))
  p_hat <- predict(ps_model, newdata = data, type = "response")

  # Trimming avoids extreme inverse-probability weights when overlap is weak
  p_hat <- pmin(pmax(p_hat, trim), 1 - trim)

  # 2) Outcome models fitted separately by treatment arm
  mu1_formula <- as.formula(
    paste("sales ~", paste(covariate_names, collapse = " + "))
  )
  mu0_formula <- mu1_formula

  outcome_model_treated <- lm(mu1_formula, data = data[data$treatment == 1, ])
  outcome_model_control <- lm(mu0_formula, data = data[data$treatment == 0, ])

  # Predicted potential outcomes for every unit
  mu1_hat <- predict(outcome_model_treated, newdata = data)
  mu0_hat <- predict(outcome_model_control, newdata = data)

  # 3) Doubly robust correction terms:
  #   residuals (Y - mu_hat) are reweighted by inverse propensity scores.
  #   If either the PS model or the outcome models are correctly specified,
  #   the estimator remains consistent.
  score_treated <- (data$treatment / p_hat) * (data$sales - mu1_hat) + mu1_hat
  score_control <- ((1 - data$treatment) /
    (1 - p_hat)) * (data$sales - mu0_hat) + mu0_hat

  mean(score_treated) - mean(score_control)
}

```

```

}

# Point estimate
aipw_ate <- estimate_aipw_ate(df_aipw, covariates, trim = 0.01)
cat(sprintf("Manual AIPW ATE estimate: $%.2f\n", aipw_ate))

```

Manual AIPW ATE estimate: \$18.85

```

# 4) Bootstrap inference for AIPW
set.seed(123)
B <- 1000
n <- nrow(df_aipw)
boot_ate <- numeric(B)

for (b in seq_len(B)) {
  idx <- sample.int(n, size = n, replace = TRUE)
  boot_data <- df_aipw[idx, ]
  boot_ate[b] <- estimate_aipw_ate(boot_data, covariates, trim = 0.01)
}

aipw_se <- sd(boot_ate)
aipw_ci <- quantile(boot_ate, probs = c(0.025, 0.975))
aipw_boot_mean <- mean(boot_ate)

```

Table 2: AIPW Bootstrap Results

Statistic	Value
Bootstrap Mean (AIPW)	19.2227
Bootstrap SE (AIPW)	8.4778
95% Percentile CI (AIPW)	[2.1545, 35.7856]

6.1 AIPW Critical Discussion Points

- Doubly robust consistency: AIPW is consistent if either the propensity score model or the outcome regression models are correctly specified.
- Efficiency: Compared with simple IPW, AIPW is typically more stable and can achieve the semiparametric efficiency bound under correct specification.
- Debiasing logic: The residual-weighted terms, $(Y - \hat{\mu})$, correct the regression predictions using inverse-probability weighting.

- Overlap/common support: Propensity scores should stay away from 0 and 1; weak overlap creates extreme weights and unstable estimates, so diagnostics and trimming are important.

Looking at where AIPW falls relative to the other ATE methods, the point estimate of \$18.85 sits closest to DML-XGBoost at \$18.35, while the linear methods — plain regression adjustment at \$15.15 and DML-Lasso at \$15.23 — come in around \$3.50 lower. This is a consistent split: both flexible methods produce higher estimates and both linear methods produce lower ones. We read this as evidence that the covariate–outcome relationship has some non-linear structure that linear regression alone misses, and the residual term $(Y - \hat{\mu})$ in the AIPW score is partially picking it up where the linear $\hat{\mu}$ falls short.

On the doubly-robust framing itself, it is worth noting that the manual IPW estimate blew up at $-\$318$ in this dataset because of the weak overlap and the resulting extreme weights. AIPW at \$18.85 sits much closer to plain regression adjustment (\$15.15) than to manual IPW, which suggests that in this dataset the regression-adjustment side of AIPW is doing most of the work. The IPW correction term is not moving the estimate much, since the underlying weights are too unstable to push the answer in any one direction. The double-robustness property therefore functions more as a safety net against gross misspecification than as a meaningful correction here.

7 Double Machine Learning (DML): PLR with Cross-Fitting

We now estimate the ATE with DoubleML using the partially linear regression model:

$$Y = \beta D + g(X) + \epsilon, \quad D = m(X) + \xi$$

Cross-fitting (sample splitting) is used to reduce overfitting bias in nuisance estimation, and the orthogonal score makes the final β less sensitive to small nuisance-model errors.

```
# Data setup for DML
x_cols <- paste0("X", 1:20)
dml_vars <- c("y", "d", x_cols)
dml_df <- casedata_task3[complete.cases(casedata_task3[, dml_vars]), dml_vars]

# Binary treatment check
stopifnot(all(unique(dml_df$d) %in% c(0, 1)))

# DoubleML data backend
dml_data <- DoubleMLData$new(
  data = dml_df,
```

```

    y_col = "y",
    d_cols = "d",
    x_cols = x_cols
  )

# Tasks for out-of-sample nuisance performance
# Use only X covariates as predictors to match DML nuisance definitions.
task_y_data <- dml_df[, c("y", x_cols)]
task_y <- TaskRegr$new(id = "outcome_task", backend = task_y_data, target = "y")

task_d_data <- dml_df[, x_cols]
task_d_data$d_factor <- factor(dml_df$d, levels = c(0, 1))
task_d <- TaskClassif$new(
  id = "treatment_task",
  backend = task_d_data,
  target = "d_factor",
  positive = "1"
)

# Helper to construct requested learners with a fallback if IDs differ by install
get_lrn <- function(primary_id, fallback_id = NULL, ...) {
  if (primary_id %in% mlr_learners$keys()) {
    return(lrn(primary_id, ...))
  }
  if (!is.null(fallback_id) && fallback_id %in% mlr_learners$keys()) {
    message("Learner ", primary_id, " not found; using fallback ", fallback_id)
    return(lrn(fallback_id, ...))
  }
  stop("Required learner not found: ", primary_id,
    if (!is.null(fallback_id))
      paste0(" (fallback also missing: ", fallback_id, ")")
    else "")
}

# Run one DML specification: nuisance metrics + PLR estimate
run_dml_spec <- function(spec_name, regr_id, classif_id,
  regr_fallback = NULL, classif_fallback = NULL) {
  set.seed(123)

  # Outcome nuisance model g(X): evaluate out-of-sample RMSE
  learner_y <- get_lrn(regr_id, regr_fallback)
  rr_y <- resample(task_y,

```

```

        learner_y$clone(),
        rsmp("cv", folds = 5),
        store_models = FALSE)
rmse_y <- rr_y$aggregate(msr("regr.rmse"))

# Selection nuisance model m(X): evaluate out-of-sample log-loss and error
learner_d_class <- get_lrn(classif_id,
                           classif_fallback,
                           predict_type = "prob")
rr_d <- resample(task_d,
                 learner_d_class$clone(),
                 rsmp("cv", folds = 5),
                 store_models = FALSE)
logloss_d <- rr_d$aggregate(msr("classif.logloss"))
ce_d <- rr_d$aggregate(msr("classif.ce"))

# PLR in DoubleML uses regression nuisance learners for both l(X) and m(X)
learner_m_regr <- get_lrn(regr_id, regr_fallback)
dml_plr <- DoubleMLPLR$new(
  data = dml_data,
  ml_l = learner_y$clone(),
  ml_m = learner_m_regr$clone(),
  n_folds = 5
)
dml_plr$fit()

data.frame(
  Model = spec_name,
  ATE_Beta = as.numeric(dml_plr$coef),
  Std_Error = as.numeric(dml_plr$se),
  RMSE_Y = as.numeric(rmse_y),
  LogLoss_D = as.numeric(logloss_d),
  Classif_Error_D = as.numeric(ce_d),
  stringsAsFactors = FALSE
)
}

# Three requested learner families
dml_lasso <- run_dml_spec(
  spec_name = "Lasso",
  regr_id = "regr.glmnet",
  classif_id = "classif.lasso",

```

```

    regr_fallback = "regr.cv_glmnet",
    classif_fallback = "classif.cv_glmnet"
)

dml_rf <- run_dml_spec(
  spec_name = "Random Forest",
  regr_id = "regr.ranger",
  classif_id = "classif.ranger"
)

dml_xgb <- run_dml_spec(
  spec_name = "XGBoost",
  regr_id = "regr.xgboost",
  classif_id = "classif.xgboost"
)

dml_results <- rbind(dml_lasso, dml_rf, dml_xgb)
dml_results$CI_Lower_95 <- dml_results$ATE_Beta - 1.96 * dml_results$Std_Error
dml_results$CI_Upper_95 <- dml_results$ATE_Beta + 1.96 * dml_results$Std_Error

cat("\n=== DML (PLR) Comparison Across Learners ===\n\n")

```

=== DML (PLR) Comparison Across Learners ===

```

dml_results_to_print <- dml_results
num_cols <- sapply(dml_results_to_print, is.numeric)
dml_results_to_print[num_cols] <- lapply(dml_results_to_print[num_cols], round, 4)
print(dml_results_to_print, row.names = FALSE)

```

	Model	ATE_Beta	Std_Error	RMSE_Y	LogLoss_D	Classif_Error_D
	Lasso	15.2253	5.7710	152.7603	0.4170	0.1928
	Random Forest	13.6981	6.1429	164.2681	0.4429	0.2040
	XGBoost	18.3478	5.8701	170.4984	0.8409	0.2206
CI_Lower_95	CI_Upper_95					
		3.9140	26.5365			
		1.6579	25.7383			
		6.8425	29.8531			


```
# "Well-fitting" model selection rule:
# prioritize low out-of-sample nuisance errors (RMSE_Y and LogLoss_D)
scale01 <- function(x) {
  rng <- range(x)
  if (diff(rng) == 0) return(rep(0, length(x)))
  (x - rng[1]) / diff(rng)
}

dml_results$fit_score <- scale01(dml_results$RMSE_Y) +
  scale01(dml_results$LogLoss_D)
best_idx <- which.min(dml_results$fit_score)

cat("\nBest-fitting nuisance combination (lower RMSE + LogLoss):",
    dml_results$Model[best_idx], "\n")
```

Best-fitting nuisance combination (lower RMSE + LogLoss): Lasso

7.1 DML Discussion Points

- DML solves regularization bias and overfitting bias: sample splitting and cross-fitting reduce overfitting from flexible learners.
- Predictive performance of nuisance models is central: lower RMSE/log-loss means cleaner residualization (orthogonalization) for treatment-effect estimation.
- Compared with weighting estimators, PLR-style partialling out avoids direct division by propensity scores, which can be more stable under weak overlap.
- Even with ML-based nuisance models, causal validity still depends on conditional independence and excluding bad controls (e.g., colliders).

Across the three nuisance learners, the PLR estimates span about \$4.65 — from \$13.70 (Random Forest) to \$18.35 (XGBoost), with Lasso in between at \$15.23. A spread that wide is not a small implementation detail; the choice of nuisance learner is moving the treatment-effect estimate by roughly a third of the smallest value. Lasso and XGBoost disagree by about \$3, and Random Forest comes in even lower than the linear method, which is the more surprising piece since RF is also a flexible learner. We do not want to overinterpret this without further diagnostics, but it is consistent with RF over-smoothing on tabular data when there are relatively few samples per leaf. Either way, the cross-learner range is wide enough that we would hesitate to read any single DML estimate as the answer.

On the selection rule, our combined RMSE + LogLoss criterion picked Lasso as the best-fitting nuisance combination, but XGBoost is the learner whose treatment-effect estimate sits closest to AIPW — i.e., closest to where the doubly-robust estimator places the answer. That is

a tension worth flagging: predictive performance on the nuisance functions and where the treatment-effect estimate ends up are not the same thing, and one does not necessarily anchor the other. “Best-fitting nuisance” is a model-fit verdict; it does not automatically translate into “most accurate treatment effect.” We use the predictive criterion as the rule for transparency, not because we believe it is decisive on its own.

8 Comparison of Treatment Effects Across Methods

```
# Extract treatment effects and standard errors from all methods
ate_regression <- coef(model_adj)["d"]
se_regression <- sqrt(diag(vcovHC(model_adj, type = "HC3")))[ "d"]

att_no_replace <- coef(model_no_replace)["d"]
se_no_replace <- sqrt(diag(vcovHC(model_no_replace, type = "HC3")))[ "d"]

att_with_replace <- coef(model_with_replace)["d"]
se_with_replace <- sqrt(diag(vcovCL(model_with_replace, cluster=~subclass)))[ "d"]

ate_euc <- coef(lm_euc)["d"]
se_euc <- sqrt(diag(vcovHC(lm_euc, type = "HC3")))[ "d"]

ate_ipw <- ate_ipw_manual
se_ipw <- ipw_boot_se

ate_ipw_lm <- coef(lm_ipw)["d"]
se_ipw_lm <- sqrt(diag(vcovHC(lm_ipw, type = "HC3")))[ "d"]

ate_aipw <- aipw_ate
se_aipw <- aipw_se

# Calculate 95% confidence intervals
ci_lower_reg <- ate_regression - 1.96 * se_regression
ci_upper_reg <- ate_regression + 1.96 * se_regression

ci_lower_no_rep <- att_no_replace - 1.96 * se_no_replace
ci_upper_no_rep <- att_no_replace + 1.96 * se_no_replace

ci_lower_rep <- att_with_replace - 1.96 * se_with_replace
ci_upper_rep <- att_with_replace + 1.96 * se_with_replace
```

```

ci_lower_euc <- ate_euc - 1.96 * se_euc
ci_upper_euc <- ate_euc + 1.96 * se_euc

ci_lower_ipw <- ipw_boot_ci[1]
ci_upper_ipw <- ipw_boot_ci[2]

ci_lower_ipw_lm <- ate_ipw_lm - 1.96 * se_ipw_lm
ci_upper_ipw_lm <- ate_ipw_lm + 1.96 * se_ipw_lm

ci_lower_aipw <- aipw_ci[1]
ci_upper_aipw <- aipw_ci[2]

# Create base comparison table for classical/weighting estimators
comparison_table_base <- data.frame(
  Method = c(
    "Regression Adjustment",
    "Matching (1:1 No Replacement)",
    "Matching (1:1 With Replacement)",
    "Euclidean Distance Matching",
    "Inverse Probability Weighting (IPW) - Manual Formula",
    "IPW Weighted Regression (Secondary)",
    "Augmented IPW (AIPW) - Manual Formula"
  ),
  Estimand = c(
    "ATE",
    "ATT",
    "ATT",
    "ATT",
    "ATE",
    "ATE",
    "ATE"
  ),
  Inference = c(
    "HC3",
    "HC3",
    "Clustered (subclass)",
    "HC3",
    "Bootstrap (1000)",
    "HC3",
    "Bootstrap (1000)"
  ),
  Estimate = c(

```

```

    round(ate_regression, 4),
    round(att_no_replace, 4),
    round(att_with_replace, 4),
    round(ate_euc, 4),
    round(ate_ipw, 4),
    round(ate_ipw_lm, 4),
    round(ate_aipw, 4)
  ),
  Std_Error = c(
    round(se_regression, 4),
    round(se_no_replace, 4),
    round(se_with_replace, 4),
    round(se_euc, 4),
    round(se_ipw, 4),
    round(se_ipw_lm, 4),
    round(se_aipw, 4)
  ),
  CI_Lower = c(
    round(ci_lower_reg, 4),
    round(ci_lower_no_rep, 4),
    round(ci_lower_rep, 4),
    round(ci_lower_euc, 4),
    round(ci_lower_ipw, 4),
    round(ci_lower_ipw_lm, 4),
    round(ci_lower_aipw, 4)
  ),
  CI_Upper = c(
    round(ci_upper_reg, 4),
    round(ci_upper_no_rep, 4),
    round(ci_upper_rep, 4),
    round(ci_upper_euc, 4),
    round(ci_upper_ipw, 4),
    round(ci_upper_ipw_lm, 4),
    round(ci_upper_aipw, 4)
  ),
  row.names = NULL
)

# Add DML learner-specific estimates to the same comparison table
dml_comparison <- data.frame(
  Method = paste0("DML-PLR (", dml_results$Model, ")"),
  Estimand = rep("ATE", nrow(dml_results)),

```

```

Inference = rep("Orthogonal score SE (cross-fit)", nrow(dml_results)),
Estimate = round(dml_results$ATE_Beta, 4),
Std_Error = round(dml_results$Std_Error, 4),
CI_Lower = round(dml_results$CI_Lower_95, 4),
CI_Upper = round(dml_results$CI_Upper_95, 4),
row.names = NULL
)

comparison_table <- rbind(comparison_table_base, dml_comparison)

```

=== Summary: Treatment Effects Across All Methods (with 95% CIs) ===

		Method	Estimand		
		Regression Adjustment		ATE	
		Matching (1:1 No Replacement)		ATT	
		Matching (1:1 With Replacement)		ATT	
		Euclidean Distance Matching		ATT	
		Inverse Probability Weighting (IPW) - Manual Formula		ATE	
		IPW Weighted Regression (Secondary)		ATE	
		Augmented IPW (AIPW) - Manual Formula		ATE	
		DML-PLR (Lasso)		ATE	
		DML-PLR (Random Forest)		ATE	
		DML-PLR (XGBoost)		ATE	
	Inference	Estimate	Std_Error	CI_Lower	CI_Upper
	HC3	15.1536	5.8029	3.7800	26.5273
	HC3	17.1503	5.8309	5.7217	28.5788
	Clustered (subclass)	23.2531	4.3893	14.6502	31.8560
	HC3	16.1581	5.8028	4.7845	27.5316
	Bootstrap (1000)	-318.1235	81.2465	-487.5767	-173.1673
	HC3	15.8064	8.3043	-0.4701	32.0829
	Bootstrap (1000)	18.8487	8.4778	2.1545	35.7856
	Orthogonal score SE (cross-fit)	15.2253	5.7710	3.9140	26.5365
	Orthogonal score SE (cross-fit)	13.6981	6.1429	1.6579	25.7383
	Orthogonal score SE (cross-fit)	18.3478	5.8701	6.8425	29.8531

Note: ATT rows are matching estimands; all other rows report ATE.

DML rows include learner-specific PLR estimates with 5-fold cross-fitting and orthogonal-score

9 Discussion

One thing worth noting upfront is that every method used here relies on the same assumption: that the 20 covariates are enough to account for all the confounding between who gets the discount and how much they buy. We can't really test this – if there's something unobserved driving both treatment and sales (e.g., customer motivation), all of these estimates could be off. But given that we have 20 covariates covering demographics and purchasing behavior, it seems like a reasonable assumption for this dataset.

9.1 1. Comparison of ATE vs. ATT Estimates

An important thing to keep in mind when comparing the results is that not all methods estimate the same thing.

- **ATE methods** (Regression, IPW, AIPW, DML) estimate the effect if the discount were given to everyone. Ignoring the broken manual IPW, these range from roughly **\$13.70 to \$18.85**.
- **ATT methods** (Matching) estimate the effect for the customers who actually got the discount, ranging from about **\$16.16 to \$23.25**.

The ATT estimates being higher than the ATE could suggest that the customers who received the discount also happen to respond more strongly to it – something like selection on returns. Though we're not 100% sure this isn't partly driven by differences in how the methods handle the data (e.g., the poor balance in the without-replacement matching might inflate things).

9.2 2. Methodological Divergence and Instability

- **IPW blowup:** The manual IPW gave $-\$318$, which is nonsense. we already discussed this above – it's an overlap problem with extreme weights. The weighted regression IPW ($\sim \$15.81$) is much more reasonable since it doesn't rely only on the weights.
- **Matching differences:** The without-replacement estimate ($\$17.15$) and with-replacement estimate ($\$23.25$) are pretty far apart. Given the balance diagnostics, we'd trust the with-replacement version more – the without-replacement matching left large SMDs on X10–X14, so those matched pairs weren't really comparable. The with-replacement version got balance mostly under control, though it comes with more variance from reusing controls.

9.3 3. The Role of Double Robustness and Machine Learning

- **AIPW (\$18.85)** is doubly robust – it stays consistent if either the propensity score model or the outcome model is right. In theory it should also be more efficient than plain IPW.
- **DML** results vary by learner:
 - XGBoost: \$18.35
 - Lasso: \$15.23
 - Random Forest: \$13.70

The fact that XGBoost and AIPW land in a similar range (~18) is interesting – it might mean there are some non-linearities in the covariate-outcome relationship that OLS misses but these methods pick up. The Random Forest estimate being lower is a bit puzzling and could reflect differences in how well the nuisance models fit.

9.4 4. Conclusion: Which Estimate to Trust?

We'd lean toward trusting the **AIPW** or **DML-XGBoost** estimates, for a few reasons:

1. They don't assume linearity the way plain regression does.
2. AIPW has the double robustness property, which gives some insurance against model misspecification.
3. They use the full dataset rather than discarding unmatched units, so they're answering the ATE question more directly.

That said, none of these methods can fully overcome the unconfoundedness assumption. If there are important unobserved confounders, all estimates could be biased.

9.4.1 Suggestions for the Marketing Manager

Across all the credible methods, the discount seems to increase sales by roughly \$15–19 per customer (ATE). The campaign appears to work. Whether it's worth expanding depends on the cost of the discount itself – if the discount costs less than \$15 per customer, broader rollout looks profitable.

For the customers currently being targeted, the effect might be even larger (~\$17–23 based on matching ATT). So the existing targeting seems to be picking customers who respond well.

If the company wants to be more confident before scaling up, running a proper randomized experiment on a sample of untargeted customers would help. That would give a clean ATE

estimate without having to worry about whether the covariates capture everything. For now, the observational evidence is encouraging but comes with the usual caveat that we're assuming the data accounts for all relevant confounders.

10 Reflection on AI Use

We used AI tools at various points during this assignment, mainly in a supporting role rather than for generating the analysis or writing from scratch.

Tools used: Primarily Google Gemini for code-related tasks, and NotebookLM for revisiting theoretical concepts from the course material.

What we used AI for:

- Debugging code when we ran into errors or unexpected output, and getting suggestions on how to improve code quality (e.g., fixing syntax issues, making sure we were using the right functions from MatchIt or DoubleML).
- Checking for oversights in our analysis – essentially using it as a second pair of eyes to see if we missed something obvious or made a methodological mistake.
- Searching for information to refresh our understanding of the theory behind the methods, particularly around IPW, AIPW, and how DML cross-fitting works.
- NotebookLM was useful for going back over course readings and lecture material to make sure we had the concepts right before writing up the discussion.

How we handled AI input: We treated AI suggestions as starting points rather than final answers. When it flagged something or suggested a change, I checked it against the course material and our own understanding before incorporating it. The analytical choices, interpretations, and written discussion are our own.